Puzzle RPA

Кейс Puzzle RPA, SK Challenge 2026

BI чат-бот с дашбордами по вопросу

Менеджер пишет вопрос на русском и получает живой дашборд прямо в переписке

Автор

Александр Корнилович

github.com/alkornilovich/Cases



Задача и ценность

Руководитель получает ответ на вопрос о продажах за секунды, без заявки аналитику и без поиска отчета в BI

Что мешает сейчас

Ответ ждут от аналитика

Нестандартный вопрос уходит в очередь, и решение принимают без цифр

Нужного отчета нет в BI

Готовые отчеты покрывают стандартные метрики, а разрез «отстающие точки Урала» в них не заложен

Код от LLM рискует данными

Модель, которая сама пишет SQL, может ошибиться в цифре или в запросе к рабочей базе

Что дает бот

Ответ за секунды

Вопрос словами превращается в KPI, фильтры и 4 графика, выгрузка данных занимает меньше секунды

Анализ продолжается диалогом

Новая колонка, другой разрез или фильтр одной фразой, без разработчика и нового отчета

Цифрам можно доверять

Считает код, модель только выбирает команду, поэтому бот не выдумывает числа и не меняет базу

Код на GitHub

Живой дашборд

Источники [1] кейс Puzzle RPA «Чат-бот с интерактивной визуализацией данных», SK Challenge 2026; [2] замер времени выгрузки на данных кейса

Три слоя решения

Модель понимает вопрос, каталог задает границу, код считает цифры. Дальше границы модель не проходит

01 Понимание

правила и бесплатная LLM

«План-факт по регионам
за последний год»

Может ошибиться

поэтому ее ответ дальше проверяется

02 Каталог

контракт с данными

дашборд
разрез = регион
метрики = план, факт
период = окт 2025 ... сен 2026

Отсекает ошибки

все, чего нет в каталоге, отбрасывается

03 Исполнение

код без генерации

```
SELECT region, sum(plan), sum(fact)
FROM sales ...
WHERE sale_date >= ?
→ 33 787 строк → дашборд
```

Считает точно

тот же код на любых данных

граница доверия

Безопасность по устройству

изолировать нечего, генерируемого кода нет

Цифры без выдумки

все числа считает код, модель их не пишет

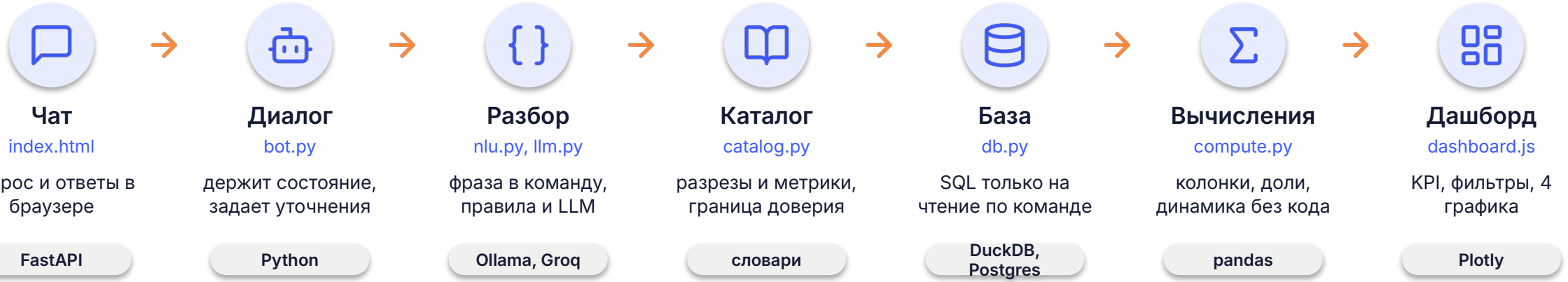
Новая база за дни

для Oracle клиента описывается каталог

Источники [2] код решения, app/nlu.py, app/catalog.py, app/db.py

Как устроен бот

Семь шагов от фразы до дашборда, каждый отдельный модуль на Python, ответ приходит меньше чем за полсекунды



Скорость ответа

0,46 с

от вопроса до готового дашборда, мой замер

2 мс	разбор фразы
200 мс	SQL и выгрузка
260 мс	упаковка для браузера

Дашборд хранится как состояние

- ✓ запрос и команда
- ✓ единая таблица с вычислениями
- ✓ настройки 4 графиков
- ✓ фильтры и переключатели
- ✓ 20 последних версий

Отсюда бесплатно сохранение, отмена шага, копия и возврат в том же виде

Решение в цифрах

2 500	строк кода в 9 файлах
73	тестовых запроса на разбор
11	проверок безопасности
2	команды для запуска

Источники [2] код решения на GitHub; замер времени на данных кейса, медиана из 5 запусков

Безопасность

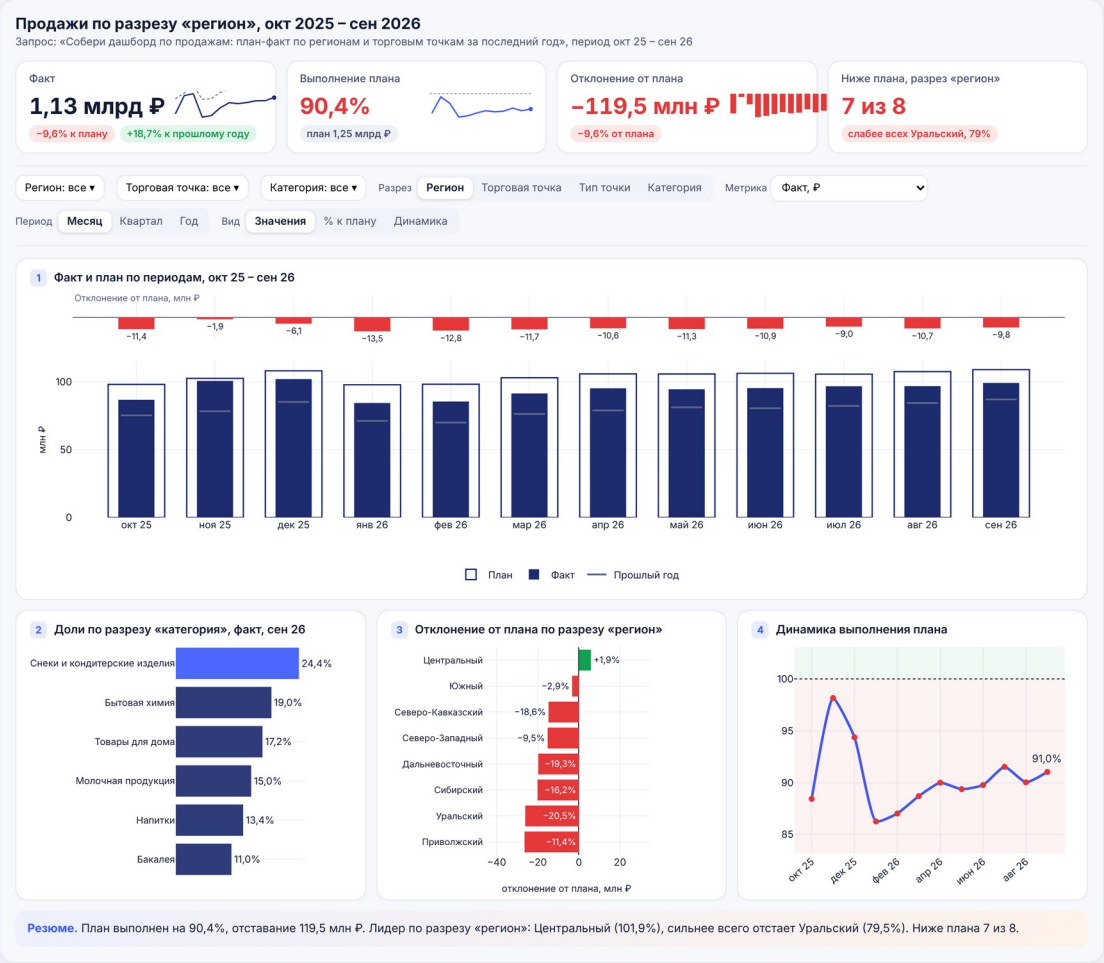
Каждое предупреждение из кейса закрыто механизмом и проверено тестом, 11 проверок из 11

Требование кейса	Как закрыто	Тест
Код не удаляет и не меняет данные	выполняются только SELECT, роль базы только на чтение	✓ DELETE и DROP отклонены
Код не ходит в сеть	после загрузки данных база отключает доступ к файлам и сети	✓ чтение файла и URL отклонено
Настройки нельзя ослабить	конфигурация базы заблокирована	✓ смена настройки отклонена
Нет SQL-инъекций	имена колонок из каталога, значения передаются параметрами	✓ DROP в тексте не попал в SQL
Код не исполняется бесконечно	таймаут 15 секунд и лимит 200 тыс. строк	✓ прерывание по таймеру
Формулы без кода	только + − × ÷, числа и колонки, деление на ноль дает пусто	✓ возведение в степень и код отклонены
Пустые данные	понятный ответ вместо ошибки, графики показывают «нет данных»	✓ запрос за 2010 год

Источники [1] раздел кейса «Ограничения и подсказки»; [2] tests/test_security.py

Дашборд в чате

Ответ на «план-факт по регионам и торговым точкам за последний год», все цифры пересчитываются при каждом фильтре



1 KPI с контекстом

факт к плану и к прошлому году, мини-графики по месяцам

2 Фильтры и переключатели

поиск по 300 точкам, мультивыбор, каскад регион → точка, месяц, квартал, год

3 Графики по стандарту IBCS

факт сплошной, план контуром, прошлый год риской, отклонения зеленым и красным

4 Резюме

считает код, текст по шаблону, модель не выдумывает цифры

Живой дашборд

Источники [2] скриншот работающего бота на данных кейса

Работа диалогом

Вычисления, фильтры и сохранение идут обычными фразами, бот показывает, что именно он сделал

«Добавь колонку с процентом выполнения плана»

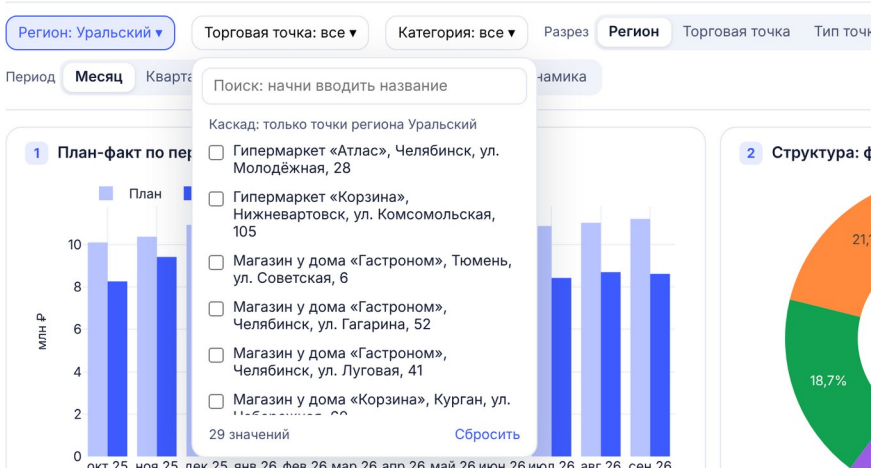
Единая таблица, сгруппированная по разрезу «Регион»

строк в разрезе 1, в единой таблице 1 730

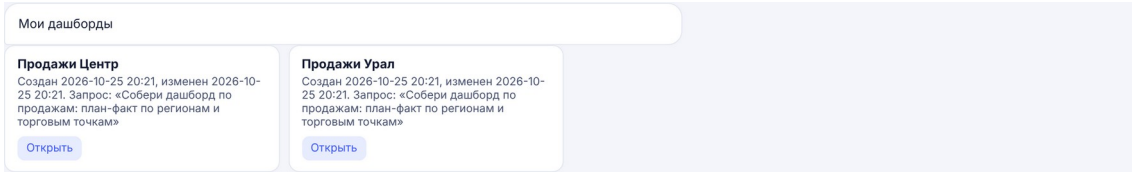
Регион	План, ₽	Факт, ₽	Количество, шт	Скидки, ₽	Сделки, шт	Выполнение плана, %	Отклонение от плана, ₽	Процент выполнения плана, %
Уральский	126 959 700	100 917 433	855 162	2 360 565	5 263	79,5	-26 042 267	79,5

Новая колонка выделена в таблице и сразу появляется в переключателе «Метрика», ее можно отменить

Каскадный фильтр с поиском



«Сохрани как Продажи Центр», «покажи мои дашборды»



Открытый дашборд восстанавливается с таблицей, вычислениями, графиками и фильтрами

Есть переименование, удаление, копия, поиск по тексту запроса и 20 версий

Источники [2] скриншоты работающего бота, сценарий из приложения к кейсу

Точность разбора

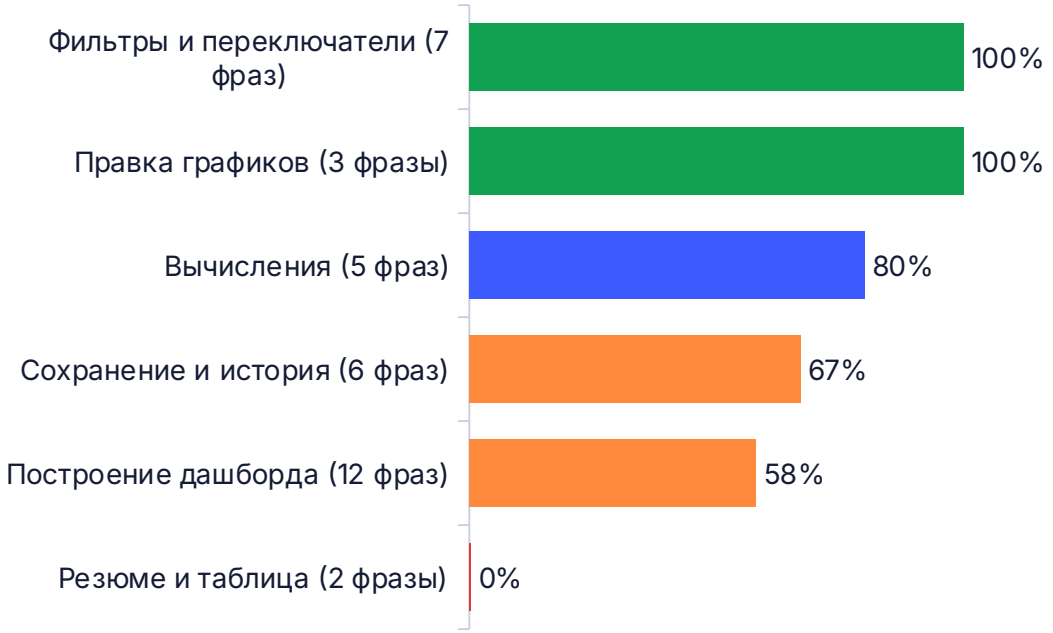
Язык кейса бот знает полностью, слабее всего первый запрос на дашборд, туда и направлена LLM

38 из 38 запросов из приложения к кейсу разобраны верно

25 из 35 новых фраз верно с первого прогона, без LLM

4 из 10 ошибок бот сам отметил как непонятные и передал бы LLM

Сравнение точности по группам команд на новых фразах



Типы ошибок и что с ними делает бот

Незнакомое слово — 5

«Выведи план и факт по молочной продукции»

LLM разбирает фразу, ответ проверяет каталог

Перепутан смысл — 4

«Какие выводы по дашборду?» запускал новый дашборд

бот показывает, как понял запрос, шаг отменяется одной фразой

Ложное совпадение — 1

«супермаркетов» находило клиента «Маркет»

поиск имен только по целым словам

Все 10 ошибок исправлены, теперь проходят 73 запроса из 73

Источники [2] tests/eval_queries.py, 38 запросов кейса и 35 новых фраз в двух отложенных выборках

Путь к клиентам Puzzle

Решение переносится в промышленный контур без переписывания логики

01

Oracle клиента

описать таблицы клиента в каталоге, SQL собирается тем же кодом

02

Корпоративный чат

подключить API бота к чат-боту Puzzle или Telegram Mini App

03

LLM в контуре

Ollama на сервере клиента, данные не покидают компанию

04

Права доступа

роль базы на пользователя, бот видит только разрешенные строки

Запустить у себя

pip install -r requirements.txt

uvicorn app.main:app --port 8000

открыть localhost:8000

Код, данные и тесты

github.com/alkornilovich/Cases

python -m tests.eval_queries

python -m tests.test_security

Источники [2] код решения на GitHub; [3] документация Ollama и Groq о бесплатном доступе к моделям